

Query Processing

Hash Join

1. Introduction

The **Hash Join** algorithm computes natural joins and equi-joins by partitioning both relations using a hash function on the join attributes.

Key Idea: If two tuples satisfy the join condition, they must have the same join attribute values, and therefore hash to the same partition. We only need to compare tuples within matching partitions.

Key Terminology

- h : Hash function mapping JoinAttrs to $\{0, 1, \dots, n_h\}$
- n_h : Number of partitions
- $r_0, r_1, \dots, r_{n_h}$: Partitions of relation r
- $s_0, s_1, \dots, s_{n_h}$: Partitions of relation s
- **Build relation:** Smaller relation used to build hash index (typically s)
- **Probe relation:** Larger relation used to probe the index (typically r)
- M : Available memory blocks
- b_b : Buffer blocks allocated per partition

1.1 Running Example

Relation	Tuples	Blocks
student (build)	5,000	$b_s = 100$
takes (probe)	10,000	$b_r = 400$

2. Hash Join Algorithm

Hash join works in two main phases:

1. **Partition Phase:** Hash both relations into n_h partitions
2. **Build & Probe Phase:** For each partition pair, build index on s_i and probe with r_i

Algorithm: Hash Join

```
/* Phase 1: Partition both relations */
for each tuple t_s in s:
    i = h(t_s[JoinAttrs])
    add t_s to partition s_i

for each tuple t_r in r:
    i = h(t_r[JoinAttrs])
    add t_r to partition r_i

/* Phase 2: Build and Probe for each partition */
for i = 0 to n_h:
    load s_i into memory
    build in-memory hash index on s_i (using hash function h2)

    for each tuple t_r in r_i:
        probe hash index to find matching t_s where
            t_s[JoinAttrs] = t_r[JoinAttrs]
        for each matching t_s:
            output join(t_r, t_s)
```

2.1 Visual Explanation

Step-by-Step Example

Given: student and takes with join on ID

Hash function: $h(\text{ID}) = \text{ID} \bmod 5$ (creates 5 partitions)

Phase 1: Partitioning

Partition	student tuples	takes tuples
$i = 0$	IDs: 00100, 00205, ...	IDs: 00100, 00205, ...
$i = 1$	IDs: 00101, 00206, ...	IDs: 00101, 00206, ...
$i = 2$	IDs: 00102, 00207, ...	IDs: 00102, 00207, ...
$i = 3$	IDs: 00103, 00128, ...	IDs: 00103, 00128, ...
$i = 4$	IDs: 00104, 00129, ...	IDs: 00104, 00129, ...

Phase 2: Build & Probe

1. Load s_0 (student partition 0) into memory
2. Build hash index on s_0 using different hash function h_2
3. For each tuple in r_0 (takes partition 0), probe index
4. Output matching pairs
5. Repeat for partitions 1, 2, 3, 4

Why two hash functions?

- h_1 : Partitions data so each partition fits in memory
- h_2 : Creates efficient in-memory hash index for fast lookups

3. Choosing the Number of Partitions

Key Constraint: Each partition s_i of the build relation (plus its hash index) must fit in memory.

$$n_h \geq \left\lceil \frac{b_s}{M} \right\rceil$$

Where b_s = blocks in build relation, M = available memory blocks.

Best Practice:

- Use the **smaller relation** as the build relation
- Add a **fudge factor** (typically 20%) to handle skew: $n_h = 1.2 \times \lceil b_s/M \rceil$

4. Recursive Partitioning

If $n_h \geq M$ (not enough memory for one buffer per partition), we need **recursive partitioning**.

Recursive Partitioning

Pass 1: Partition input into (M-1) partitions
Pass 2: For each partition still too large:
 Re-partition using a DIFFERENT hash function
Pass 3: Repeat until each partition fits in memory

When is Recursive Partitioning NOT Needed?

Recursive partitioning is avoided if:

$$M > \sqrt{b_s}$$

Example:

- Memory = 12 MB = 3,072 blocks (4 KB each)
- Maximum relation size without recursion: 3072^2 blocks = 36 GB
- A 1 GB relation needs only $\sqrt{256K} \approx 512$ blocks ≈ 2 MB memory

5. Handling Hash Table Overflow

Overflow occurs when:

- Many tuples have the same join attribute value
- Hash function produces skewed (non-uniform) partitions

5.1 Overflow Resolution

Detect overflow during build phase and handle it:

1. If partition s_i is too large, **re-partition** it using a different hash function
2. Similarly re-partition the matching r_i
3. Join only the matching sub-partitions

5.2 Overflow Avoidance

Prevent overflow proactively:

1. Initially create **many small partitions**

2. **Combine** small partitions that fit together in memory
3. Partition probe relation the same way

Fallback: If a partition has too many tuples with identical join values (cannot be split further), use **block nested-loop join** for that partition.

6. Cost Analysis

6.1 Case 1: No Recursive Partitioning

Cost Formula: Basic Hash Join

Block Transfers:

$$3(b_r + b_s) + 4n_h$$

Breakdown:

1. Partition phase: Read both relations = $b_r + b_s$
2. Partition phase: Write all partitions = $b_r + b_s$
3. Build & Probe phase: Read all partitions = $b_r + b_s$
4. Partially filled blocks overhead: $\leq 4n_h$

Disk Seeks:

$$2 \left(\left\lceil \frac{b_r}{b_b} \right\rceil + \left\lceil \frac{b_s}{b_b} \right\rceil \right) + 2n_h$$

Example: student $\bowtie$ takes

Given: $M = 20$ blocks, $b_s = 100$, $b_r = 400$, $b_b = 3$

Number of partitions: $n_h = \lceil 100/20 \rceil = 5$

Block Transfers:

$$3(100 + 400) = 3 \times 500 = \mathbf{1500} \text{ transfers}$$

Seeks:

$$2 \left(\left\lceil \frac{100}{3} \right\rceil + \left\lceil \frac{400}{3} \right\rceil \right) = 2(34 + 134) = \mathbf{336} \text{ seeks}$$

6.2 Case 2: With Recursive Partitioning

When $M < \sqrt{b_s}$, recursive partitioning is needed.

Cost Formula: Recursive Hash Join

Number of partitioning passes:

$$P = \left\lceil \log_{\lfloor M/b_b \rfloor - 1} \left(\frac{b_s}{M} \right) \right\rceil$$

Block Transfers:

$$2(b_r + b_s) \times P + (b_r + b_s)$$

Disk Seeks:

$$2 \left(\left\lceil \frac{b_r}{b_b} \right\rceil + \left\lceil \frac{b_s}{b_b} \right\rceil \right) \times P$$

6.3 Case 3: Build Relation Fits in Memory

Best Case: In-Memory Hash Join

If the entire build relation fits in memory ($b_s \leq M$):

$$\text{Block Transfers} = b_r + b_s \quad \text{Seeks} = 2$$

No partitioning needed! ($n_h = 0$)

1. Load entire build relation into memory
2. Build hash index
3. Stream probe relation and output matches

7. Hybrid Hash Join

An optimization when memory is larger than needed for partitioning but cannot hold the entire build relation.

Key Insight: Keep partition s_0 in memory during partitioning; probe it immediately without writing to disk.

Algorithm: Hybrid Hash Join

```
/* Partitioning s (build relation) */
Allocate (n_h + 1) * b_b blocks for partition buffers
Use remaining (M - (n_h + 1) * b_b) blocks to keep s_0 in memory
Build hash index on s_0 as tuples arrive

/* Partitioning r (probe relation) */
For tuples hashing to partition 0:
    Immediately probe s_0's hash index
    Output matching tuples
    Discard r_0 tuples (no need to store)

For tuples hashing to partitions 1 to n_h:
    Write to disk as usual

/* Join remaining partitions */
For i = 1 to n_h:
    Perform standard hash join on (r_i, s_i)
```

Savings:

- Partition s_0 : Never written to disk, never read back
- Partition r_0 : Never written to disk, never read back
- Saves $2 \times (\text{size of } s_0 + r_0)$ block transfers

Hybrid Hash Join Example

Given:

- Build relation: 5 GB
- Memory: 1 GB
- Block size: 4 KB, $b_b = 20$

Without Hybrid:

- Need $n_h \approx 5000/1000 = 5$ partitions
- All partitions written and read back

With Hybrid:

- $s_0 \approx 1$ GB stays in memory
- Only partitions s_1 to s_4 (≈ 4 GB) written to disk
- **Savings:** $\approx 20\%$ of I/O cost

When is Hybrid useful?

$$M \gg \sqrt{b_s \times b_b}$$

8. Comparison: Hash Join vs Other Methods

Algorithm	Advantages	Disadvantages
Hash Join	$O(b_r + b_s)$ cost; works well for large relations	Requires partitioning; overflow handling
Merge Join	Single pass if sorted; good for sorted data	Requires sorted input or sorting cost
Indexed Nested-Loop	Fast for small outer relation with index	Expensive for large outer relations

When to choose Hash Join:

- Relations are large and unsorted
- No useful indices exist
- Memory is sufficient for partitioning (ideally $M > \sqrt{b_s}$)
- Join selectivity is unknown (hash join is robust)

9. Summary

Key Takeaways

- **Hash Join** partitions both relations using hash function on join attributes
- **Two phases:** Partition (write to disk) $\rightarrow$ Build & Probe (join partitions)
- **Build relation** should be smaller; must fit one partition in memory
- **Cost:** $3(b_r + b_s)$ transfers without recursion
- **Recursive partitioning** needed when $M < \sqrt{b_s}$
- **Best case:** Build fits in memory $\rightarrow$ only $b_r + b_s$ transfers, 2 seeks
- **Hybrid hash join** keeps s_0 in memory, saving $\approx 20\%$ I/O
- **Overflow handling:** Re-partition or fall back to nested-loop join